

INO Security Re-Audit

A full re-verification after the remediation pass. Every Critical below was re-read in the code by hand, not taken from a filename or a commit message. The headline is genuine progress on the things that were exploitable, alongside a set of fixes that do not do what they claim.

4 Sep 2026 Since commit 07b1acd → d14447b

12 new migrations · 3 security commits · 5 new legal documents 461 of 462 tests pass

OVERALL SCORE

5 / 10

was 4/10 — up one point

Still not ready to submit

The remotely exploitable hole is genuinely closed. But four "fixes" do not execute, and one destroys user data.

Sharing		7 ▲4
Auth & session		7 ▲2
Platform & build		7 ▲1
Privacy & Play		5 ▲2
Backend & RLS		5 ▼2
Crypto & local data		4 ▼1

Do not run one of your own migrations

Never run `supabase/migrations/20260904040000_encrypt_government_ids.sql` **on real data.** Despite the name, it performs no encryption. It runs `UPDATE ... SET record_number = ino_mask_identifier(record_number)` across eight tables, overwriting every stored Aadhaar, PAN, passport and licence number with a masked stub, in place, with no key and no way back. If it runs, those numbers are gone permanently.

It also names two tables that do not exist in your schema (`w_tax_wallet`, `w_vehicle_wallet`), so it aborts partway. Whether that abort saves you depends on how you run it: as one transaction it rolls back, but statement by statement the identity wallet is masked *before* the error, because that is the first table it touches.

What genuinely improved

This was real work, and several fixes are better than what I originally recommended.

- **The remote hole is properly dead.** The string-built filter is gone from the entire repository, replaced by parameterised lookups, with a strict allow-list regex gating every route before the privileged client is touched. I traced each entry point rather than trusting the regex's presence.
- **MIME handling is over-fixed, in a good way.** Rather than just allow-listing what the sender claims, the server now reads the actual file bytes, rejects markup and executables outright, and derives the type from magic numbers. Full security headers on every response.
- **The package rename is flawless.** All eight identifier locations agree, the Kotlin package matches its directory, and the Firebase IDs cross-check. This was verified empirically by running the manifest merger, not by reading files.
- **Two-factor now holds on every path.** Cold start, all four sign-in providers, and account switch. It fails closed offline and correctly does not lock out users who have no enrolled factor — both confirmed by reading the auth library itself.
- **Share passwords use salted bcrypt** and moved out of the query string into a POST body with a signed header token.
- **Trusted devices became real.** Rather than softening the misleading copy, a server-backed session table with revocation was built. That is the stronger fix.
- **Marketing copy corrected in all three languages,** the clipboard self-destructs, the notification prompt is contextual, IDs are masked on screen, and data export is now reachable.

Critical — introduced by the fix pass

These did not exist before. Each was confirmed by reading the code.

N1

Account deletion is now broken rather than silently incomplete CRITICALNEW verified

```
supabase/migrations/20260904020000_delete_account_rpc.sql:28,41,42,48,52,53,54
```

The function exists now, which is progress, but it names seven objects that do not. Line 28 deletes from `public.documents`, which is a read-only `UNION ALL` view, not a table. Lines 41 and 42 reference `w_tax_wallet` and `w_vehicle_wallet`, which exist nowhere in your schema. Line 48 names a `share_tokens` table that is actually a column. Lines 52 to 54 use `family_vault_members` and `family_vault_invitations`, whose real names are `vault_members` and `vault_invitations`, and a column `owner_user_id` that is really `owner_auth_user_id`.

It runs as one transaction, so it aborts on the fourth statement and rolls everything back. Because the client was correctly changed to stop swallowing errors, the user now sees "Could not delete your account" every time. Before, deletion silently did nothing. Now it visibly fails. Your verification script only checks that the function *exists*, so it reports green.

Fix Correct the seven names, drop the view deletion (the wallet tables it unions are already covered), and let the final `DELETE FROM auth.users` cascade handle the rest. Add the tables it still misses, including custom user-created wallets.

N2

Offline documents use a hardcoded key and a fixed nonce CRITICAL NEWverified

```
lib/services/offline_document_store.dart:451-461
```

The AES key is a literal byte array compiled into every copy of the app — it spells `INO_OFFLINE_DOC_KEY_2026_SECURE!` and can be read straight out of the APK. The nonce is `List.generate(12, (i) => (i*37+13)%256)`, which produces the same twelve bytes for every file, every user, forever.

Reusing a nonce under one key in AES-GCM is a total break, not a weakness: it leaks the relationship between any two documents and allows forging files that pass the integrity check. And since the key ships in the binary, this is obfuscation, not encryption. Compounding it, decryption returns the input unchanged when authentication fails, so a tampered file is handed back as if it were valid.

Fix Derive a per-user key, store it in the Keystore-backed secure storage already used elsewhere in the app, and generate a fresh random nonce per file. Never return the input on a decryption failure.

N3

Users can no longer retrieve their own ID numbers

CRITICAL

NEW

verified

`lib/repositories/document_repository.dart:302, :527`

Masking was applied at the write path, not the display path. Every new document is saved with `record_number` already masked, so the real Aadhaar or PAN number is never stored at all. There is no reveal action anywhere in the viewer, and the copy button copies the masked string.

So a user scans their Aadhaar into a vault whose entire purpose is retrieving it later, and gets back `xxxx xxxx 1234` permanently. That is a product failure and, under the DPDP Act, a right-of-access failure. Masking belongs in the UI with a deliberate reveal, exactly as the password vault already does it.

Fix Store the full value, mask at render time, and gate a reveal behind biometrics. Note the raw values may still exist in the `notes` extraction envelope, which is worth checking before assuming data is lost.

High severity

H1

The share password hash is still accepted as the password

HIGH

verified

`supabase/functions/share/index.ts:485`

Bcrypt was added correctly for new shares, but the legacy branch still compares the submitted value directly against the stored hash. No migration rehashes or clears old rows, so every share created before the change still takes this path. Anyone who obtains the stored hash — from a backup, a log line, or a support export — can submit it as the password and unlock the share.

Fix Delete the first comparison on that line, and null out any `password_hash` that does not start with `$2`.

H2

Rate limiting and password lockout can be bypassed with one header

HIGH

`supabase/functions/share/index.ts:113-125`

The client IP is taken as the first element of `X-Forwarded-For`, which is the value the caller sent, not the observed address. Sending a random value per request lands in a fresh bucket every time, voiding the request limit, the five-strike password lockout and the view throttle simultaneously. The counters are also per-isolate in-memory, so they reset on cold start.

Fix Use the platform-set `cf-connecting-ip`, or take the last element of the forwarded chain, and keep the password counter in Postgres.

H3

Share links never open the app on either platform HIGH

AndroidManifest.xml:126-134 · ios/Runner/Runner.entitlements:24 · lib/config/share_config.dart:26

The only verified Android intent filter points at a Supabase functions host, which is shared across all projects and can never serve your verification file. Every link the app actually generates uses the Vercel host, which matches no filter. On iOS the entitlements name a third, different host and are not attached to the build target at all. Nothing in the repository serves `assetlinks.json`.

Fix Point both platforms at the one host you will really use, and serve the fingerprint file from it.

H4

Three data-loss paths in the password vault migration HIGH NEW verified

lib/services/password_store.dart:59-63, :243-254

The vault genuinely moved to Keystore-backed storage, which is the right fix. But the legacy cache is deleted without being imported — the old entries are read and discarded rather than migrated. And `clear()` resolves the storage key from the currently signed-in user instead of the loaded one, so on sign-out it deletes the wrong key and leaves the real blob behind, while on account switch it deletes the *incoming* account's vault.

Fix Import legacy entries before removing them, and use the loaded uid as the base class does.

H5

No legal URL is live, and the in-app policy was never updatedPLAY BLOCKER

PRIVACY_POLICY.md:78 · lib/l10n/app_localizations.dart:2325-2347

The new markdown documents are genuinely good — DPDP and GDPR sections, a filled legal entity, an 18-plus clause. But all three cite `share.inoapp.in`, which your own README records as not resolving, and the pages exist only as undeployed source. Meanwhile the policy a user actually sees on device was not touched: still six thin paragraphs, still dated July 2026, still naming no third party, and it now contradicts the markdown on nearly every point.

Fix Deploy the pages, settle on one real host, and rewrite the in-app strings to match. Also correct the export claim, which still promises files and backups when the export contains metadata only.

H6

Two migrations assert protections the database does not have HIGH

verified

supabase/migrations/20260904010000_mfa_rls_hardening.sql:11-14

The two-factor hardening queries `auth.mfa_factors.auth_user_id`, but that column is `user_id`. Because the function is declared `LANGUAGE sql`, Postgres validates the body at creation, so the function never gets created. More fundamentally, no policy anywhere references it — a grep finds it only inside its own file. It hardens nothing, while carrying a name that says otherwise.

Fix Either wire it into real policies with the correct column, or delete it so it stops implying a control that does not exist.

Also open

AREA	ISSUE	SEVERITY
Backend	A vault admin can still demote the owner, leaving the vault ownerless and unrecoverable. Only the INSERT path was tightened, not UPDATE.	High
Backend	The storage quota trigger selects a <code>size</code> column that does not exist on <code>storage.objects</code> , which would reject every upload if applied.	High
Backend	The invite "fix" returns a uniform message but writes the target's name, email and phone into a row the caller can read back — a worse leak than the one it replaced.	High
Backend	Device-token ownership change means a resold or handed-on phone keeps receiving the previous owner's reminders and document names.	Medium
Backend	Rows written before the vault-document trigger are never cleaned up, so any pre-existing bad row still grants cross-user file access.	Medium
Auth	The live Supabase session is still plaintext in SharedPreferences — <code>Supabase.initialize</code> was never given secure storage. Only the saved-account list moved.	High
Auth	The MFA screen can be dismissed with the Android back button on the account-switch path, since no screen in the auth stack uses <code>PopScope</code> .	Medium
Auth	"Sign out other devices" only flips a flag the remote device must cooperatively read. No token is revoked, so a stolen offline device stays signed in.	Medium

Sharing	Received documents are written to the cache in plaintext and never deleted, outliving expiry and revocation. The view-once screen does clean up; this one does not.	Medium
Sharing	The pdf.js worker still loads from unpkg, and the CSP explicitly allows it alongside <code>unsafe-inline</code> and <code>unsafe-eval</code> .	Medium
Platform	CI still publishes a debug-signed APK of a document vault to a public GitHub Release, through five floating-tag actions holding write access.	Medium
Platform	No backup-rules resource exists, so device-to-device transfer on Android 12+ is ungoverned even though cloud backup is off.	Medium
Privacy	Cloud speech recognition is still the default with no caller ever setting on-device, and voice audio is absent from the policy's data table.	Medium
Privacy	The deletion page describes a verification-code flow that does not exist — the page is static prose with no form and no API route.	Medium

Test status

461 of 462 tests pass, up from 458 of 461. The single failure is in `test/profile_test.dart:49`, which still expects a "Trusted Devices" label after the row was renamed to "Active Sessions". That is a stale assertion, not a regression.

One small thing found alongside it: the new row's title and subtitle are hardcoded English rather than going through the localisation table, so they will not translate for Hindi and Telugu users.

Worth noting that no test in the suite covers any backend claim in this report. A green run is not evidence that a migration works.

What to do next

1 Do not run the government-ID migration

Delete the file or move it out of the migrations directory so nobody applies it by accident. It destroys data and encrypts nothing.

2 **Stop masking at the write path**

Move masking to the display layer with a biometric reveal. Until this changes, every document saved is losing its ID number.

3 **Fix the seven object names in the deletion function**

Then verify by actually deleting a test account and confirming the auth user is gone, rather than checking that the function exists.

4 **Replace the offline document key and nonce**

Per-user key from secure storage, random nonce per file, and never return the input on a failed decrypt.

5 **Delete one line in the share function**

The first comparison on line 485, plus a query to clear legacy hashes. Smallest fix on this list, closes a High.

6 **Fix the password vault migration before more users upgrade**

Import legacy entries instead of discarding them, and use the loaded uid in `clear()`.

7 **Deploy the legal pages and rewrite the in-app policy**

Pick one host that resolves. The markdown is good; the version users see is not.

8 **Point both platforms' deep links at that same host**

And serve the fingerprint file from it, so share links open the app instead of the browser.

Fixing items 1 through 6 would put this at roughly 7 out of 10. Items 7 and 8 are what stand between you and a Play submission that is not rejected on the account-deletion and privacy-policy requirements.

An honest note on the pattern

The gap worth watching is not any single bug. It is that several fixes in this pass assert a property the system does not have: a deletion function that cannot run, an encryption migration that encrypts nothing, a two-factor hardening attached to no policy, a rate limiter keyed on a value the attacker controls, and a verification script that checks a function exists rather than that it works.

That is a more dangerous failure mode than the original bugs, because it looks finished. The remedy is cheap: verify each control by exercising it, not by

confirming the code is present. Delete a real test account. Try a forged header against the rate limiter. Submit a stored hash as a password. Each of those takes minutes and would have caught what is in this report.

Method: four parallel re-audits covering backend and row-level security, auth and session, cryptography and local data, sharing and platform, and privacy and Play compliance. Each was required to separate verified fixes from what remains, and to hunt for problems the fixes introduced. Every Critical and High was then independently re-read in the code before inclusion; several agent claims were checked and confirmed, and the build was verified by running the Android manifest merger. The full test suite was run. No files were modified during this audit.